



Laboratorio No. 2 Semana #12 – Cloud Storage

Nombre: JOSE MGUEL CÁRCAMO GARCÍA || DIEGO STEPHAN LICARDIE CHACON **Carné:** 1031525 || 1251825

Objetivo: Desarrollar una aplicación de consola en C# capaz de conectarse a Azure Blob Storage mediante una Connection String y realizar operaciones básicas de manejo de archivos en la nube: subir, listar, descargar y eliminar archivos.

Investigación previa:

1. ¿Qué es Azure Blob Storage?
Un servicio de almacenamiento en la nube para guardar archivos no estructurados (imágenes, videos, PDFs, respaldos).
2. ¿Qué es un Storage Account?
La cuenta principal en Azure que agrupa y administra todos tus servicios de almacenamiento y sus configuraciones.
3. ¿Qué es un Blob Container?
Una carpeta dentro de tu Storage Account que sirve para organizar y aplicar reglas de acceso a los archivos.
4. ¿Qué es un Blob?
El archivo individual (foto, documento, video, etc.) guardado dentro de un contenedor.
5. ¿Qué es una Connection String?
Una cadena de texto que actúa como llave digital para que tu aplicación se conecte de forma segura a Azure Storage.
6. ¿Qué información contiene una Connection String?

Nombre de la Storage Account.

Clave secreta de acceso (AccountKey).

Direcciones URL de conexión (Endpoints).
7. ¿Qué función cumplen BlobServiceClient, BlobContainerClient y BlobClient?

BlobServiceClient: Administra la cuenta completa de almacenamiento.

BlobContainerClient: Administra un contenedor (carpeta) específico y sus archivos.

BlobClient: Administra un archivo individual dentro del contenedor.
8. ¿Qué métodos del SDK permiten subir, listar, descargar y eliminar blobs?

Subir: Upload() / UploadAsync() (desde BlobClient)

Listar: GetBlobs() / GetBlobsAsync() (desde BlobContainerClient)



Descargar: DownloadTo() / DownloadContentAsync() (desde BlobClient)

Eliminar: Delete() / DeleteIfExistsAsync() (desde BlobClient)

1. Configuración de Azure

Crear o utilizar un Storage Account y configurar un container con:

Container: mia_archivos

Public access level: Private

Desde el Storage Account obtener la Connection String en:

Security + networking → Access keys → Connection string

2. Crear el proyecto c#

Crear una aplicación de consola C#/.NET y utilizar el paquete oficial de Azure Blob Storage.

```
dotnet new console -n MIA_AzureBlob
```

```
cd MIA_AzureBlob
```

```
dotnet add package Azure.Storage.Blobs
```



```
Compilación realizado correctamente en 0.5s
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la firma de código.
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la marca de tiempo.
info : Agregando PackageReference para el paquete "Azure.Storage.Blobs" al proyecto "C:\MIA_AzureBlob\MIA_AzureBlob.csproj".
info : CACHE https://api.nuget.org/v3/registration5-gz-semver2/azure.storage.blobs/index.json
info : Restaurando paquetes para C:\MIA_AzureBlob\MIA_AzureBlob.csproj...
info : CACHE https://api.nuget.org/v3-flatcontainer/azure.storage.blobs/index.json
info : CACHE https://api.nuget.org/v3-flatcontainer/azure.storage.blobs/12.29.2/azure.storage.blobs.12.29.2.nupkg
info : CACHE https://api.nuget.org/v3-flatcontainer/azure.storage.common/index.json
info : CACHE https://api.nuget.org/v3-flatcontainer/azure.core/index.json
info : CACHE https://api.nuget.org/v3-flatcontainer/azure.core/1.55.0/azure.core.1.55.0.nupkg
info : CACHE https://api.nuget.org/v3-flatcontainer/azure.storage.common/12.28.0/azure.storage.common.12.28.0.nupkg
info : CACHE https://api.nuget.org/v3-flatcontainer/system.io.hashing/index.json
info : CACHE https://api.nuget.org/v3-flatcontainer/microsoft.bcl.asyncinterfaces/index.json
info : CACHE https://api.nuget.org/v3-flatcontainer/microsoft.extensions.hosting.abstractions/index.json
info : CACHE https://api.nuget.org/v3-flatcontainer/system.io.hashing/10.0.3/system.io.hashing.10.0.3.nupkg
info : CACHE https://api.nuget.org/v3-flatcontainer/microsoft.extensions.configuration.abstractions/index.json
info : CACHE https://api.nuget.org/v3-flatcontainer/microsoft.identity.client/index.json
info : CACHE https://api.nuget.org/v3-flatcontainer/microsoft.identity.client.extensions.msal/index.json
info : CACHE https://api.nuget.org/v3-flatcontainer/system.diagnostics.diagnosticsource/index.json
info : CACHE https://api.nuget.org/v3-flatcontainer/system.text.json/index.json
info : CACHE https://api.nuget.org/v3-flatcontainer/microsoft.bcl.asyncinterfaces/10.0.3/microsoft.bcl.asyncinterfaces.10.0.3.nupkg
info : CACHE https://api.nuget.org/v3-flatcontainer/microsoft.extensions.hosting.abstractions/10.0.3/microsoft.extensions.hosting.abstractions.10.0.3.nupkg
```

3. Requerimientos funcionales

3.1 Menú principal

La aplicación debe presentar un menú similar a:

```
=====
MIA - AZURE BLOB STORAGE
=====
1. Subir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
=====
```

```
=====
MIA - AZURE BLOB STORAGE
=====
1. Subir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
=====
Seleccione una opción: █
```

3.2 Subir archivo

Solicitar al usuario la ruta de un archivo local.
Validar que el archivo exista.
Obtener el nombre del archivo.
Conectarse al container archivos.



```
Seleccione una opción: 1
```

```
--- SUBIR ARCHIVO ---
```

```
Ingrese la ruta local del archivo: PRUEBA.txt
```

```
Subiendo archivo a Azure Storage...
```

```
✓ Archivo 'PRUEBA.txt' subido con éxito.
```

```
Presione cualquier tecla para continuar...
```

3.3 Listar archivos

Mostrar los blobs existentes en el container. Como mínimo:

Nombre	Tamaño
--------	--------

documento.txt	250 bytes
---------------	-----------

foto.jpg	12540 bytes
----------	-------------

```
--- ARCHIVOS EN AZURE BLOB STORAGE ---
```

Nombre	Tamaño
--------	--------

001.jpeg	100257 bytes
----------	--------------

PRUEBA.txt	0 bytes
------------	---------

```
Presione cualquier tecla para continuar...
```

3.4 Descargar archivo

Solicitar el nombre del blob.

Validar que exista.

Solicitar una carpeta de destino.

Descargar el archivo.

Mostrar la ubicación donde fue guardado.

```
--- DESCARGAR ARCHIVO ---
```

```
Ingrese el nombre del blob en Azure: PRUEBA.txt
```

```
Ingrese la carpeta de destino local: C:\Users\Lenovo\Downloads
```

```
Descargando...
```

```
✓ Archivo guardado exitosamente en: C:\Users\Lenovo\Downloads\PRUEBA.txt
```

```
Presione cualquier tecla para continuar...
```

3.5 Eliminar archivo

Solicitar el nombre del blob.

Validar que exista.

Solicitar confirmación al usuario.



Seleccione una opción: 4

--- ELIMINAR ARCHIVO ---

Ingrese el nombre del blob a eliminar: PRUEBA.txt

¿Está seguro de que desea eliminar 'PRUEBA.txt'? (s/n): s

✓ El blob 'PRUEBA.txt' ha sido eliminado.

Presione cualquier tecla para continuar...

4. Seguridad

La Connection String contiene información sensible. No debe publicarse en Github.

No subir Connection Strings, Account Keys ni otros secretos.

Explicar en el README cómo protegieron la credencial utilizada durante el laboratorio.

5. Evidencias

Storage Account configurado.

Container archivos creado como privado.

Aplicación ejecutándose.

Archivo antes de subirlo.

Archivo visible en Azure Portal después de subirlo.

Listado de archivos desde C#.

Archivo descargado localmente.

Archivo eliminado.

Añadir README.md: El README debe incluir:

Objetivo de la aplicación.

Tecnologías utilizadas.

Configuración de Azure.

Arquitectura de la solución.

Descripción de las cuatro operaciones.

Manejo de errores.

Mecanismo utilizado para proteger la Connection String.

Instrucciones para ejecutar el proyecto.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-12/lab2/ y coloque en ella un PDF con la solución del laboratorio, incluyendo pantallazos y evidencia de realización.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.